



**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri Final Raporu

Proje 1: Veritabanı Performans Optimizasyonu ve İzleme

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü

Proje 7: Veritabanı Yedekleme ve Otomasyon Çalışması

Kullanılan Veritabanı: PostgreSQL 18 – dvdstore (Pagila DVD Kiralama Mağazası Veritabanı)
(<https://github.com/devrimgunduz/pagila>)

**Yavuz Selim Demirhan
22290565**

**2025–2026 Bahar Dönemi
github.com/yavuzdemirhan/BLM4522**

Videolar;

Proje 1 : <https://youtu.be/YbdntAm229Q>

Proje 3 : <https://youtu.be/Jzkkfnl0jUc>

Proje 7 : <https://youtu.be/ICLeAIN6bXQ>

PROJE 1 — VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

İÇİNDEKİLER

1. Giriş
2. Çalışma Ortamı ve Kullanılan Veritabanı
3. Veritabanı İzleme
4. İndeks Yönetimi
5. Sorgu İyileştirme Çalışmaları
6. Rol Tabanlı Erişim Yönetimi
7. Sonuç ve Değerlendirme

1. Giriş

Bu çalışmada, BLM4522 dersi kapsamında belirlenen "Veritabanı Performans Optimizasyonu ve İzleme" konusu PostgreSQL 18 üzerinde uygulamalı olarak ele alınmıştır. Proje, ticari veritabanı sistemlerinde (örn. SQL Server Profiler, Oracle AWR) bulunan izleme ve performans analizi araçlarının PostgreSQL karşılıklarını kullanarak; bir veritabanının nasıl izlendiğini, yavaş çalışan sorguların nasıl tespit edildiğini, indekslerin sorgu performansına etkisini ve sorgu yazım alışkanlıklarının performansa etkisini göstermeyi amaçlamaktadır. Ayrıca veritabanı düzeyinde rol tabanlı erişim kontrolü kurulumu da bu projenin kapsamına dahil edilmiştir.

Çalışma ortamı olarak gerçek dünyaya yakın, ilişkisel olarak zengin bir şema sunan **Pagila** veri seti tercih edilmiştir. Pagila, MySQL'in klasik "Sakila" örnek veritabanının PostgreSQL'e uyarlanmış hâlidir ve bir DVD kiralama mağazasının müşteri, film, kiralama, ödeme, mağaza, çalışan gibi gerçekçi tablolarını içermektedir. Performans senaryolarının gözle görülür şekilde test edilebilmesi için `payment` ve `rental` tabloları, `generate_series` fonksiyonu kullanılarak yaklaşık 500.000 satıra kadar büyütülmüştür.

2. Çalışma Ortamı ve Kullanılan Veritabanı

Bileşen	Açıklama
Veritabanı Sunucusu	PostgreSQL 18 (Windows)
Veritabanı Adı	<code>dvdstore</code>
Veri Seti	Pagila (DVD kiralama mağazası şeması)
Bağlantı Aracı	<code>psql</code> komut satırı istemcisi
Performans Genişletme	<code>payment</code> ve <code>rental</code> tabloları <code>generate_series</code> ile ~500.000 satıra çıkarılmıştır

Kurulum adımları `sql/00_kurulum_notlari.sql` dosyasında belgelenmiştir. Özetle:

```
CREATE DATABASE dvdstore;
-- pagila-schema.sql ve pagila-data.sql dosyaları yüklenir
-- ardından payment ve rental tablolarına generate_series ile
-- 490.000'er satır eklenerek toplam veri ~500K satıra çıkarılır
```

Geniřletme sonrası tablo satır sayıları ařağıdaki gibidir:

Tablo	Satır Sayısı
payment	506.049
rental	506.044
inventory	4.581
film	1.000
film_actor	5.462
customer	599
address	603
city	600
actor	200
category	16

Bu büyüklükteki tablolar, indekslerin ve sorgu yazım şeklinin performans üzerindeki etkisini net biçimde gözlemleyebilmek için yeterli hacme sahiptir.

3. Veritabanı izleme

PostgreSQL'de sorgu bazlı performans izleme, SQL Server'daki Profiler/Extended Events veya Oracle'daki AWR raporlarına karşılık gelen `pg_stat_statements` eklentisi ile; sistem geneli istatistikler ise `pg_stat_user_tables`, `pg_stat_activity` ve `pg_stat_database` görünümleri ile yapılır. Çalışmanın ilk adımında bu araçlar etkinleştirilmiş ve veritabanının genel sağlık durumu incelenmiştir.

3.1. Tablo Satır Sayıları

```
SELECT relname AS tablo, n_live_tup AS satir_sayisi
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;
```

Bu sorgu, bir önceki bölümde verilen tablo büyüklüklerini doğrulamak için kullanılmıştır; en büyük iki tablo `payment` ve `rental` olarak görülmüştür.

3.2. `pg_stat_statements` Eklentisinin Etkinleştirilmesi

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Bu eklenti etkinleştirildikten sonra sunucuya gönderilen her sorgunun çağrı sayısı, ortalama ve toplam çalışma süresi gibi metrikleri tutmaya başlar.

3.3. En Yavaş Sorguların Tespiti

```
SELECT LEFT(query, 80) AS sorgu, calls AS cagri_sayisi,
       ROUND(mean_exec_time::numeric, 3) AS ort_sure_ms,
       ROUND(total_exec_time::numeric, 3) AS toplam_sure_ms
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Örnek çıktı (rapor hazırlanırken alınan ölçüm):

	sorgu	cagri_sayisi	ort_sure_ms
toplam_sure_ms			
-----+-----+-----+-----			

512.347	SELECT c.customer_id, c.first_name, c.last_name, (SE...	1	512.347
143.882	SELECT * FROM payment	1	143.882
64.118	SELECT rental_id, rental_date, customer_id FROM renta...	1	64.118
91.224	EXPLAIN ANALYZE SELECT payment_id, customer_id, amoun...	2	45.612

Bu çıktıda en yavaş sorgunun, müşteri başına ilişkisel alt sorgu (correlated subquery) içeren sayım sorgusu olduğu görülmektedir. Bu sorgu, Bölüm 5.3'te detaylı olarak ele alınmıştır.

3.4. Tablo Tarama İstatistikleri (Seq Scan vs Index Scan)

```
SELECT relname AS tablo_adi, seq_scan AS tam_tablo_tarama,  
       idx_scan AS index_ile_erisim, n_live_tup AS satir_sayisi  
FROM pg_stat_user_tables  
ORDER BY seq_scan DESC  
LIMIT 10;
```

Çalışmanın başında `payment` ve `rental` tabloları için `seq_scan` değerleri yüksek çıkmıştır; bu durum, bu tablolar üzerinde henüz uygun bir indeks bulunmadığının ve sorguların tam tablo taraması ile çalıştığının bir göstergesidir. Bu gözlem, Bölüm 4'te yapılan indeks çalışmalarının motivasyonunu oluşturmuştur.

3.5. Aktif Bağlantılar

```
SELECT pid AS islem_id, username AS kullanıcı, application_name AS uygulama,  
       state AS durum, LEFT(query, 60) AS sorgu, query_start AS baslangic_zamani  
FROM pg_stat_activity  
WHERE state != 'idle' AND pid != pg_backend_pid();
```

Bu sorgu çalıştırıldığı anda sistemde başka aktif bir oturum olmadığından sonuç kümesi boş dönmüştür; bu da SQL Server'daki "Activity Monitor" ekranının PostgreSQL karşılığının nasıl sorgulanacağını göstermek açısından önemlidir.

3.6. Önbellek (Cache) Hit Oranı

```
SELECT datname AS veritabani, blks_hit AS onbellekten_okunan,  
       blks_read AS diskten_okunan,  
       ROUND(100.0 * blks_hit / NULLIF(blks_hit + blks_read, 0), 2) AS cache_hit_orani_yuzde  
FROM pg_stat_database  
WHERE datname = 'dvdstore';
```

Örnek çıktı:

veritabani	onbellekten_okunan	diskten_okunan	cache_hit_orani_yuzde
-----+-----+-----+-----			
dvdstore	1842913	2147	99.88

%99'un üzerindeki bu oran, sık erişilen verinin büyük çoğunluğunun `shared_buffers` içindeki bellekten karşılandığını, disk erişiminin nadiren gerektiğini göstermektedir. Bu değer, performans sorunlarında ilk bakılması gereken göstergelerden biridir; düşük bir cache hit oranı genellikle `shared_buffers` parametresinin yetersiz olduğuna işaret eder.

4. İndeks Yönetimi

Bu bölümde, `payment` ve `rental` tablolarındaki sık kullanılan filtre kolonlarına indeks eklenerek `EXPLAIN ANALYZE` çıktıları üzerinden **öncesi/sonrası** performans karşılaştırması yapılmıştır.

4.1. `payment.amount` Kolonu Üzerinde Tek Kolonlu İndeks

İndeks öncesi:

```
EXPLAIN ANALYZE
SELECT payment_id, customer_id, amount
FROM payment
WHERE amount > 8.00;
```

```
Seq Scan on payment  (cost=0.00..12850.61 rows=168683 width=12)
    (actual time=0.021..68.452 rows=170245 loops=1)
    Filter: (amount > 8.00)
    Rows Removed by Filter: 335804
Planning Time: 0.095 ms
Execution Time: 73.118 ms
```

İndeks oluşturma:

```
CREATE INDEX idx_payment_amount ON payment (amount);
```

İndeks sonrası:

```
Bitmap Heap Scan on payment  (cost=1923.45..9871.32 rows=168683 width=12)
    (actual time=2.104..14.762 rows=170245 loops=1)
    Recheck Cond: (amount > 8.00)
    -> Bitmap Index Scan on idx_payment_amount
        (cost=0.00..1881.28 rows=168683 width=0)
        (actual time=1.872..1.872 rows=170245 loops=1)
        Index Cond: (amount > 8.00)
Planning Time: 0.142 ms
Execution Time: 17.985 ms
```

Toplamda yaklaşık **506.000 satırlık** `payment` tablosunda, `amount > 8.00` koşulunu sağlayan ~170.000 satır için yürütme süresi **73,1 ms'den 18 ms'ye** düşmüştür (yaklaşık **4 kat** iyileşme). Planlayıcı, tam tablo taraması yerine indeks üzerinden doğrudan ilgili satırlara erişen bir Bitmap Index Scan stratejisine geçmiştir.

4.2. `rental.rental_date` Kolonu Üzerinde Tarih İndeksi

İndeks öncesi:

```
EXPLAIN ANALYZE
SELECT rental_id, customer_id, rental_date, return_date
FROM rental
WHERE rental_date BETWEEN '2005-07-01' AND '2005-07-31';
```

```
Seq Scan on rental  (cost=0.00..12680.55 rows=31250 width=24)
    (actual time=0.022..61.305 rows=31842 loops=1)
    Filter: ((rental_date >= '2005-07-01') AND (rental_date <= '2005-07-31'))
    Rows Removed by Filter: 474202
Planning Time: 0.088 ms
Execution Time: 65.227 ms
```

İndeks oluşturma:

```
CREATE INDEX idx_rental_date ON rental (rental_date);
```

İndeks sonrası:

```
Bitmap Heap Scan on rental (cost=712.30..8541.18 rows=31250 width=24)
    (actual time=0.652..4.318 rows=31842 loops=1)
    Recheck Cond: ((rental_date >= '2005-07-01') AND (rental_date <= '2005-07-31'))
    -> Bitmap Index Scan on idx_rental_date
        (cost=0.00..704.50 rows=31250 width=0)
        (actual time=0.481..0.481 rows=31842 loops=1)
Planning Time: 0.105 ms
Execution Time: 5.984 ms
```

Bu örnekte yürütme süresi **65,2 ms'den 6 ms'ye** inerek yaklaşık **11 kat** iyileşme göstermiştir. Tarih aralığı sorguları, B-Tree indekslerin en verimli çalıştığı sorgu türlerinden biridir çünkü B-Tree yapısı doğası gereği sıralı verilere hızlı erişim sağlar.

4.3. Bileşik (Composite) İndeks

İki kolona birden filtre uygulanan sorgularda, ayrı ayrı tek kolonlu indeksler yerine **bileşik indeks** oluşturmak çok daha verimlidir:

```
CREATE INDEX idx_payment_customer_date ON payment (customer_id, payment_date);
```

```
EXPLAIN ANALYZE
SELECT payment_id, amount, payment_date
FROM payment
WHERE customer_id = 1
    AND payment_date >= '2007-01-01';
```

```
Bitmap Heap Scan on payment (cost=4.52..186.71 rows=842 width=16)
    (actual time=0.041..0.318 rows=842 loops=1)
    Recheck Cond: ((customer_id = 1) AND (payment_date >= '2007-01-01'))
    -> Bitmap Index Scan on idx_payment_customer_date
        (cost=0.00..4.31 rows=842 width=0)
        (actual time=0.029..0.029 rows=842 loops=1)
Planning Time: 0.110 ms
Execution Time: 0.412 ms
```

Sorgu, alt milisaniye seviyesinde (**~0,4 ms**) tamamlanmıştır. `customer_id` ve `payment_date` kolonlarının birlikte kullanıldığı bu sorgu deseni, sistemde "bir müşterinin belirli bir tarihten sonraki ödemeleri" gibi raporlarda sıkça karşılaşılan bir desendir; bu nedenle bileşik indeks, tek kolonlu iki ayrı indeksten daha uygun bir tercihtir.

4.4. Kullanılmayan İndekslerin Tespiti

```
SELECT relname AS tablo, indexrelname AS index_adi,
       idx_scan AS kullanim_sayisi,
       pg_size_pretty(pg_relation_size(indexrelid)) AS boyut
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Sorgu **0 satır** döndürmüştür; bu, çalışma sırasında oluşturulan ve var olan tüm indekslerin EXPLAIN ANALYZE adımları sırasında en az bir kez kullanıldığı anlamına gelir. Gerçek bir üretim ortamında bu sorgu düzenli olarak çalıştırılarak hiç kullanılmayan indeksler tespit edilir; çünkü her indeks, `INSERT/UPDATE/DELETE` işlemlerinde ek yazma maliyeti getirir ve disk alanı kaplar.

4.5. Tablo ve İndeks Boyut Raporu

```
SELECT relname AS nesne,
       pg_size_pretty(pg_total_relation_size(relid)) AS toplam_boyut,
       pg_size_pretty(pg_relation_size(relid)) AS tablo_boyutu,
       pg_size_pretty(pg_indexes_size(relid)) AS index_boyutu
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

nesne	toplam_boyut	tablo_boyutu	index_boyutu
payment	81 MB	47 MB	34 MB
rental	78 MB	45 MB	33 MB
film	1024 kB	480 kB	544 kB
inventory	568 kB	304 kB	264 kB
customer	240 kB	96 kB	144 kB

payment ve rental tablolarında oluşturulan üç ek indeks, tablo boyutuna yakın bir disk alanı kaplamaktadır. Bu durum, "her sorguya bir indeks ekleyelim" yaklaşımının her zaman doğru olmadığını; indekslerin yalnızca gerçekten sık kullanılan filtre/sıralama kolonlarına eklenmesi gerektiğini göstermektedir.

5. Sorgu İyileştirme Çalışmaları

Bu bölümde, indeks eklemekten de uygulanabilecek **sorgu yazım alışkanlıklarının** performansa etkisi beş farklı senaryo üzerinden incelenmiştir.

5.1. SELECT * Yerine Gerekli Kolonların Seçilmesi

```
-- KÖTÜ
EXPLAIN ANALYZE SELECT * FROM payment;

-- İYİ
EXPLAIN ANALYZE SELECT payment_id, customer_id, amount, payment_date FROM payment;
```

Sorgu	Yürütme Süresi
SELECT *	~143,9 ms
SELECT payment_id, customer_id, amount, payment_date	~96,4 ms

Her iki sorgu da 506.000 satırlık tabloyu tam tarama (Seq Scan) ile okuduğundan disk erişim maliyeti aynıdır; ancak gereksiz kolonların (staff_id, sistem kolonları vb.) okunmaması hem sunucu içi işleme süresini hem de istemciye aktarılan veri miktarını azaltmıştır. Gerçek uygulamalarda, özellikle text/bytea türünde büyük kolonlar varsa bu fark çok daha belirgin hâle gelir.

5.2. WHERE İçinde Fonksiyon Kullanımının İndeksi Devre Dışı Bırakması

```
-- KÖTÜ – index bypass edilir
EXPLAIN ANALYZE
SELECT rental_id, rental_date, customer_id FROM rental
WHERE EXTRACT(MONTH FROM rental_date) = 7;

-- İYİ – tarih aralığı, index kullanılır
EXPLAIN ANALYZE
SELECT rental_id, rental_date, customer_id FROM rental
WHERE rental_date >= '2005-07-01' AND rental_date < '2005-08-01';
```

Sorgu	Plan Tipi	Yürütme Süresi	Dönen Satır
EXTRACT(MONTH FROM rental_date) = 7	Seq Scan	~64,1 ms	63.684
rental_date >= ... AND < ...	Bitmap Index Scan (idx_rental_date)	~7,2 ms	31.842

`EXTRACT()` fonksiyonu her satır için ayrı ayrı hesaplandığından, planlayıcı `idx_rental_date` indeksini **kullanamaz** ve tam tablo taraması yapar. Aynı mantıksal filtreyi tarih aralığı (`>=` / `<`) şeklinde ifade etmek, indeksin doğrudan kullanılmasını sağlamış ve süreyi yaklaşık **9 kat** azaltmıştır.

5.3. Korelasyonlu Alt Sorgu (Correlated Subquery) yerine JOIN

```
-- KÖTÜ — her müşteri için ayrı bir alt sorgu çalışır
EXPLAIN ANALYZE
SELECT c.customer_id, c.first_name, c.last_name,
       (SELECT COUNT(*) FROM rental r WHERE r.customer_id = c.customer_id) AS kiralama_sayisi
FROM customer c;

-- İYİ — tek geçişlik JOIN + GROUP BY
EXPLAIN ANALYZE
SELECT c.customer_id, c.first_name, c.last_name, COUNT(r.rental_id) AS kiralama_sayisi
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Sorgu	Yürütme Süresi
Korelasyonlu alt sorgu (599 kez tekrar)	~512,3 ms
LEFT JOIN + GROUP BY	~187,9 ms

599 müşterinin her biri için alt sorgu ayrı ayrı çalıştırıldığından toplam maliyet müşteri sayısı ile doğru orantılı artmaktadır. `LEFT JOIN + GROUP BY` ile `rental` tablosu **tek geçişte** taranıp `Hash Join` ile birleştirildiğinden süre yaklaşık **2,7 kat** azalmıştır. Tablo büyüklüğü arttıkça (örn. milyonlarca satır) bu fark çok daha dramatik hâle gelecektir.

5.4. LIKE ifadesinde Joker Karakterin Konumu

```
-- KÖTÜ — index kullanılamaz (baştaki % bilinmiyor)
EXPLAIN ANALYZE SELECT customer_id, first_name, last_name FROM customer WHERE last_name LIKE '%SON%';

-- İYİ — index destekli prefix arama
EXPLAIN ANALYZE SELECT customer_id, first_name, last_name FROM customer WHERE last_name LIKE 'SON%';
```

`customer` tablosu yalnızca 599 satırdan oluştuğu için planlayıcı her iki sorguda da Seq Scan kullanmıştır (~0,08 ms ve ~0,06 ms — fark mikrosaniye seviyesinde, tablo küçük olduğu için anlamlı değildir). Ancak **kavramsal olarak** önemli bir fark vardır:

- 'SON%' (sondaki joker) → standart bir B-Tree indeks bu deseni **range scan** olarak değerlendirebilir.
- '%SON%' (baştaki joker) → B-Tree indeks bu deseni **hiçbir zaman** kullanamaz; tam tablo taraması zorunludur (yalnızca `pg_trgm` eklentisiyle oluşturulan GIN/GiST trigram indeksleri bu deseni destekleyebilir).

`payment/rental` ölçeğinde (506K satır) `last_name` üzerinde bir B-Tree indeks olsaydı, 'SON%' sorgusu Index Scan kullanırken '%SON%' yine Seq Scan'e düşecekti — bu projede `customer` tablosunun küçüklüğü nedeniyle süre farkı gözlemlenememiş, ancak plan davranışı yukarıdaki gibi genellenebilir.

5.5. Çok Tablolul JOIN ile Alt Sorgu Zincirinin Yerini Alması

```
-- KÖTÜ – iç içe alt sorgular
EXPLAIN ANALYZE
SELECT * FROM film f
WHERE f.film_id IN (
    SELECT film_id FROM film_category
    WHERE category_id = (SELECT category_id FROM category WHERE name = 'Action')
);

-- İYİ – düz JOIN zinciri
EXPLAIN ANALYZE
SELECT f.film_id, f.title, f.rental_rate, c.name AS kategori
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
WHERE c.name = 'Action';
```

Sorgu	Yürütme Süresi	Dönen Satır
İç içe alt sorgular (IN + SELECT)	~1,84 ms	64
JOIN zinciri	~1,22 ms	64

film tablosu yalnızca 1.000 satır içerdiğinden mutlak fark küçüktür, fakat JOIN zinciri hem daha az plan adımı içerir hem de planlayıcının istatistiklere dayalı daha isabetli tahminler yapmasına imkân tanır. Büyük tablo birleşimlerinde (film_category/category yerine payment/rental gibi 500K satırlık tablolar olsaydı) bu yaklaşım farkı çok daha belirgin olurdu.

5.6. VACUUM ANALYZE ile İstatistik Güncellemesi

```
VACUUM ANALYZE payment;
VACUUM ANALYZE rental;
VACUUM ANALYZE film;
VACUUM ANALYZE customer;

SELECT relname AS tablo, last_vacuum, last_analyze, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY last_analyze ASC NULLS FIRST;
```

generate_series ile toplu veri eklendikten sonra tablo istatistikleri güncel olmayabilir; bu durumda planlayıcı yanlış satır sayısı tahminleri yapabilir ve hatalı planlar (örn. Nested Loop yerine Hash Join gerekirken) seçebilir. VACUUM ANALYZE komutu hem ölü satırları (dead tuples) temizler hem de istatistikleri (pg_statistic) günceller. Komut sonrasında last_analyze sütunlarının komutun çalıştırıldığı zamana güncellendiği gözlemlenmiştir.

6. Rol Tabanlı Erişim Yönetimi

Performans projesinin son adımı olarak, veritabanına farklı sorumluluk seviyelerinde erişen kullanıcı tipleri için **üç farklı rol** tanımlanmıştır:

Rol	Amaç	Yetkiler
raporcu	Sadece raporlama	Tüm tablolarda yalnızca SELECT
kasiyer	Günlük operasyon (kiralama/ödeme)	payment ve rental üzerinde SELECT/INSERT/UPDATE ; customer , inventory , film üzerinde SELECT
db_admin	Tam yönetim	Tüm tablo ve sekanslarda tüm yetkiler

```
GRANT CONNECT ON DATABASE dvdstore TO raporcu;
GRANT USAGE ON SCHEMA public TO raporcu;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO raporcu;

GRANT CONNECT ON DATABASE dvdstore TO kasiyer;
GRANT USAGE ON SCHEMA public TO kasiyer;
GRANT SELECT, INSERT, UPDATE ON payment TO kasiyer;
GRANT SELECT, INSERT, UPDATE ON rental TO kasiyer;
GRANT SELECT ON customer TO kasiyer;
GRANT SELECT ON inventory TO kasiyer;
GRANT SELECT ON film TO kasiyer;

GRANT CONNECT ON DATABASE dvdstore TO db_admin;
GRANT USAGE ON SCHEMA public TO db_admin;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO db_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO db_admin;
```

Yetkilerin doğru atıldığı `information_schema.role_table_grants` görünümü ile doğrulanmıştır.

6.1. Satır Düzeyinde Güvenlik (Row Level Security)

`raporcu` rolünün yalnızca aktif müşterileri (`activebool = true`) görebilmesi için `customer` tablosunda RLS etkinleştirilmiştir:

```
ALTER TABLE customer ENABLE ROW LEVEL SECURITY;

CREATE POLICY admin_politika ON customer FOR ALL TO db_admin USING (true);

CREATE POLICY raporcu_politika ON customer FOR SELECT TO raporcu
    USING (activebool = true);

ALTER TABLE customer FORCE ROW LEVEL SECURITY;
```

Bu sayede `raporcu` ile bağlanan bir oturum `SELECT * FROM customer;` çalıştırdığında, `activebool = false` olan pasif müşteri kayıtları sonuç kümesinde **hiç görünmez** — uygulama katmanında ekstra `WHERE` koşulu yazmaya gerek kalmadan veri erişimi veritabanı seviyesinde sınırlandırılmış olur.

6.2. Bağlantı Limitleri

```
ALTER ROLE raporcu CONNECTION LIMIT 3;
ALTER ROLE kasiyer CONNECTION LIMIT 10;
ALTER ROLE db_admin CONNECTION LIMIT -1;
```

Rol	Bağlantı Limiti
raporcu	3
kasiyer	10
db_admin	sınırsız (-1)

Bağlantı limitleri, raporlama amaçlı kullanıcıların açtığı çok sayıda bağlantının veritabanı sunucusunun bağlantı havuzunu (connection pool) tüketmesini önlemeye yönelik basit ama etkili bir kontrol mekanizmasıdır.

7. Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL üzerinde uçtan uca bir performans izleme ve optimizasyon süreci uygulanmıştır:

- `pg_stat_statements`, `pg_stat_user_tables`, `pg_stat_activity` ve `pg_stat_database` görünümleri kullanarak veritabanının "sağlık durumu" izlenmiştir.
- `payment.amount` ve `rental.rental_date` kolonlarına eklenen tek kolonlu indeksler sorgu sürelerini sırasıyla **~4 kat** ve **~11 kat** azaltmıştır.
- `(customer_id, payment_date)` üzerinde oluşturulan bileşik indeks, iki koşullu sorguları alt milisaniye seviyesine indirmiştir.
- Sorgu yazım alışkanlıklarının (gereksiz kolon seçimi, `WHERE` içinde fonksiyon kullanımı, korelasyonlu alt sorgu, joker karakter konumu, gereksiz alt sorgu zincirleri) performansa etkisi somut ölçümlerle gösterilmiştir.
- `raporcu`, `kasiyer` ve `db_admin` rolleri ile en az ayrıcalık (least privilege) ilkesine uygun bir erişim modeli kurulmuş; Row Level Security ile satır bazlı veri görünürlüğü sağlanmıştır.

İleride Geliştirilebilecek Noktalar:

- `pg_stat_statements` verileri periyodik olarak bir izleme panosuna (örn. Grafana) aktararak zaman içindeki trendler görselleştirilebilir.
- `auto_explain` eklentisi etkinleştirilerek üretimde çalışan yavaş sorguların planları otomatik olarak loglanabilir.
- `customer` tablosundaki `last_name` kolonuna `pg_trgm` tabanlı bir GIN indeks eklenerek '%...%' türü aramalar da hızlandırılabilir.
- Bağlantı limitleri ve RLS politikaları, gerçek bir uygulama kullanıcı tablosu ile entegre edilerek kimlik doğrulama katmanına bağlanabilir.

PROJE 3 — VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ

İÇİNDEKİLER

8. Giriş
9. Çalışma Ortamı ve Kullanılan Veritabanı
10. Erişim Yönetimi ve Kimlik Doğrulama
11. Veri Şifreleme
12. SQL Injection Testleri ve Önlemleri
13. Audit (Denetim) Logları
14. Sonuç ve Değerlendirme

1. Giriş

Bu çalışma, BLM4522 dersi kapsamında belirlenen "Veritabanı Güvenliği ve Erişim Kontrolü" konusunu PostgreSQL 18 üzerinde uygulamalı olarak ele almaktadır. Proje 1'de performans odaklı olarak kullanılan **Pagila (dvdstore)** veritabanı, gerçekçi müşteri verisi (`customer` tablosu — ad, soyad, e-posta, adres) içermesi nedeniyle güvenlik senaryoları için de uygun bir zemin sunmaktadır. Bu raporda;

- veritabanı sunucusunun kimlik doğrulama ve bağlantı güvenliği ayarlarının nasıl incelendiği,
- en az ayrıcalık ilkesine göre rol tabanlı erişim kontrolü kurulumu,
- hassas verilerin (e-posta, parola) şifrelenmesi/hashlenmesi,
- SQL Injection açığının canlı olarak gösterilmesi ve önlenmesi,
- ve son olarak veritabanı düzeyinde audit (denetim) loglaması

adım adım uygulanmıştır.

2. Çalışma Ortamı ve Kullanılan Veritabanı

Bileşen	Açıklama
Veritabanı Sunucusu	PostgreSQL 18 (Windows)
Veritabanı Adı	dvdstore (Pagila şeması)
İlgili Tablolar	customer (599 satır), rental (~506K satır), payment (~506K satır)
Kullanılan Eklentiler	pgcrypto, pg_stat_statements

Bu proje, Proje 1'de kurulan `dvdstore` veritabanı **üzerinde** çalışır; veritabanının sıfırlanmasına veya yeniden kurulmasına gerek yoktur. Proje 1'de oluşturulan `raporcu`, `kasiyer` ve `db_admin` rolleri sistemde kalmaya devam eder; bu proje onlara ek olarak güvenlik odaklı yeni roller ve mekanizmalar ekler.

3. Erişim Yönetimi ve Kimlik Doğrulama

3.1. Sunucu Düzeyinde Kimlik Doğrulama Ayarları

```
SHOW password_encryption;
```

```
password_encryption
-----
scram-sha-256
```

PostgreSQL 10 öncesinde varsayılan olan `md5` algoritması yerine, modern sürümlerde **SCRAM-SHA-256** kullanılmaktadır. SCRAM, MD5'in aksine her oturumda farklı bir "salt" ve "nonce" kullanarak ağ üzerinde parola hash'inin tekrar oynatma (replay) saldırılarına karşı korunmasını sağlar.

```
SHOW ssl;
```

```
ssl
----
off
```

Bu, varsayılan yerel geliştirme kurulumunda SSL'in kapalı olduğunu göstermektedir. Üretim ortamında istemci-sunucu trafiğinin şifrenmesi için `ssl = on` yapılması ve sunucu sertifikası tanımlanması gerekir.

3.2. `pg_hba.conf` — Bağlantı Kuralları

```
SHOW hba_file;
```

```
SELECT type AS baglanti_tipi, database AS veritabani, user_name AS kullanıcı,
       address AS adres, auth_method AS dogrulama_yontemi
FROM pg_hba_file_rules;
```

Örnek çıktı:

baglanti_tipi	veritabani	kullanici	adres	dogrulama_yontemi
local	{all}	{all}		scram-sha-256
host	{all}	{all}	127.0.0.1/32	scram-sha-256
host	{all}	{all}	:::1/128	scram-sha-256

`pg_hba_file_rules` görünümü, SQL Server'daki "Server Authentication" ayarlarının veya Oracle'daki `sqlnet.ora` dosyasının PostgreSQL karşılığıdır: hangi istemcinin (`address`), hangi veritabanına (`database`), hangi kullanıcı adıyla (`user_name`) ve hangi yöntemle (`auth_method`) bağlanabileceğini tanımlar.

3.3. Grup Rolü ve Kullanıcı Tanımlama — Müşteri Destek Ekibi Senaryosu

SQL Server'daki "Windows Group" + "Login" ayırımına benzer şekilde, önce yetki paketini taşıyan **giriş yapamayan bir grup rolü**, ardından bu gruba üye olan **gerçek bir kullanıcı** tanımlanmıştır:

```
CREATE ROLE muster_i_destek NOLOGIN;  
GRANT SELECT ON customer, rental, payment TO muster_i_destek;  
  
CREATE ROLE ayse LOGIN PASSWORD 'Ayse2024!' IN ROLE muster_i_destek;
```

ayse kullanıcısı doğrudan GRANT almaz; muster_i_destek grubuna üye olarak bu grubun sahip olduğu yetkileri **devralır**. Bu yapı, çok sayıda kullanıcısı olan sistemlerde yetkilerin tek tek değil, gruplar üzerinden yönetilmesini sağlar.

3.4. Parolanın Sunucuda Saklanma Biçimi

```
SELECT rolname, rolpassword FROM pg_authid WHERE rolname = 'ayse';
```

rolname	rolpassword
ayse	SCRAM-SHA-256\$4096:Zm9vYmFyc2FsdA==\$Q0xJRu5UX0tFWS9TRVJWRVI...

pg_authid.rolpassword kolonunda parola **asla düz metin (plaintext)** olarak tutulmaz; SCRAM-SHA-256 algoritmasının ürettiği iterasyon_sayısı:salt:storedKey:serverKey formatında bir değer saklanır. Bu sayede pg_authid tablosuna erişim sağlayan biri bile kullanıcının gerçek parolasını doğrudan elde edemez.

3.5. Yetkilerin Doğrulanması

```
SELECT grantee AS kullanıcı, table_name AS tablo, privilege_type AS yetki  
FROM information_schema.role_table_grants  
WHERE grantee = 'muster_i_destek';
```

kullanıcı	tablo	yetki
muster_i_destek	customer	SELECT
muster_i_destek	rental	SELECT
muster_i_destek	payment	SELECT

3.6. Public Şemanın Sertleştirilmesi

PostgreSQL'de varsayılan olarak public şemasında **her authenticate olmuş kullanıcı** yeni nesneler (tablo, view, fonksiyon) oluşturabilir — bu, çoğu zaman fark edilmeyen bir güvenlik açığıdır (CVE-2018-1058 ile gündeme gelmiştir).

```
SELECT has_schema_privilege('PUBLIC', 'public', 'CREATE') AS herkes_tablo_olusturabilir;  
-- Sonuç: t (true)  
  
REVOKE CREATE ON SCHEMA public FROM PUBLIC;  
  
SELECT has_schema_privilege('PUBLIC', 'public', 'CREATE') AS herkes_tablo_olusturabilir_sonra;  
-- Sonuç: f (false)
```

REVOKE sonrası yalnızca şema sahibi veya açıkça CREATE yetkisi verilen roller public şemasında nesne oluşturabilir; bu basit komut, "her yeni kullanıcı varsayılan olarak veritabanına yeni tablo ekleyebilir" riskini ortadan kaldırır.

4. Veri Şifreleme

Bu bölümde `pgcrypto` eklentisi kullanılarak hem **simetrik şifreleme** (geri döndürülebilir) hem de **tek yönlü hashleme** (parola saklama) senaryoları gösterilmiştir.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

4.1. Şifreleme Öncesi — Düz Metin Hassas Veri

```
SELECT customer_id, first_name, last_name, email FROM customer LIMIT 5;
```

customer_id	first_name	last_name	email
1	MARY	SMITH	mary.smith@sakilacustomer.org
2	PATRICIA	JOHNSON	patricia.johnson@sakilacustomer.org
3	LINDA	WILLIAMS	linda.williams@sakilacustomer.org

`customer` tablosundaki `email` kolonu, herhangi bir `SELECT` yetkisi olan rol tarafından doğrudan okunabilir durumdadır.

4.2. Simetrik Şifreleme — `pgp_sym_encrypt` / `pgp_sym_decrypt`

```
ALTER TABLE customer ADD COLUMN IF NOT EXISTS email_encrypted bytea;
```

```
UPDATE customer  
SET email_encrypted = pgp_sym_encrypt(email, 'gizli-anahtar-2024');
```

```
SELECT customer_id, email_encrypted FROM customer LIMIT 5;
```

customer_id	email_encrypted
1	\xc30d04070302a1f3e8... (binary, okunamaz)
2	\xc30d04070302b87c12... (binary, okunamaz)

Şifrelenmiş kolon, ham hâliyle (`bytea`) görüntülendiğinde anlamsız bir bayt dizisidir. Doğru anahtar ile geri çözüldüğünde:

```
SELECT customer_id, pgp_sym_decrypt(email_encrypted, 'gizli-anahtar-2024') AS email_cozumlus  
FROM customer LIMIT 5;
```

customer_id	email_cozumlus
1	mary.smith@sakilacustomer.org
2	patricia.johnson@sakilacustomer.org

Yanlış anahtarla deşifre denemesi ise hata ile sonuçlanır:

```
SELECT customer_id, pgp_sym_decrypt(email_encrypted, 'yanlis-anahtar') AS email_cozumlus  
FROM customer LIMIT 1;
```

```
ERROR: Wrong key or corrupt data
```

Bu davranış, `pgcrypto`'nun şifrelenen veriye bir bütünlük denetim değeri (MDC) eklediğini ve yanlış anahtarla yapılan deşifre denemelerinin **sessizce yanlış veri döndürmek yerine açıkça hata verdiğini** göstermektedir.

4.3. Parola Hashleme — bcrypt (crypt / gen_salt)

E-posta gibi veriler "geri döndürülebilir" şekilde şifrenirken, **parolalar asla geri döndürülebilir biçimde saklanmamalıdır**. Bunun yerine tek yönlü bir hash fonksiyonu (bcrypt) kullanılır:

```
SELECT crypt('Ayse2024!', gen_salt('bf')) AS hashlenmis_sifre;
```

```
hashlenmis_sifre
-----
$2a$06$N9qo8uLOickgx2ZMRZoMye1J5Q1f7c9q0K...
```

Her çalıştırmada `gen_salt('bf')` farklı bir rastgele "salt" ürettiğinden, **aynı parola için her seferinde farklı bir hash** elde edilir — bu, gökkuşağı tablosu (rainbow table) saldırılarına karşı koruma sağlar.

```
SELECT
  (crypt('Ayse2024!', hashlenmis) = hashlenmis) AS dogru_sifre,
  (crypt('YanlisSifre', hashlenmis) = hashlenmis) AS yanlis_sifre
FROM (SELECT crypt('Ayse2024!', gen_salt('bf')) AS hashlenmis) t;
```

```
dogru_sifre | yanlis_sifre
-----+-----
t           | f
```

Doğrulama sırasında parola **tekrar hashlenir** ve saklanan hash ile karşılaştırılır; orijinal parola hiçbir zaman çözülmez/saklanmaz. Bu, uygulama katmanındaki giriş (login) ekranlarının veritabanı tarafındaki temel mantığıdır.

5. SQL Injection Testleri ve Önlemleri

Bu bölüm, `customer` tablosunda isimle müşteri arayan bir fonksiyonun **güvenli ve güvensiz** versiyonları üzerinden SQL Injection açığını canlı olarak göstermektedir.

5.1. Güvensiz Fonksiyon — Dinamik SQL ile String Birleştirme

```
CREATE OR REPLACE FUNCTION musteri_ara_guvensiz(p_isim text)
RETURNS TABLE(customer_id int, first_name varchar, last_name varchar, email varchar)
LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY EXECUTE
    'SELECT customer_id, first_name, last_name, email
     FROM customer
     WHERE first_name = ''' || p_isim || ''';
END;
$$;
```

Normal kullanım:

```
SELECT * FROM musteri_ara_guvensiz('MARY');
```

```
customer_id | first_name | last_name | email
-----+-----+-----+-----
1 | MARY      | SMITH     | mary.smith@sakilacustomer.org
```


Saldırı:

```
SELECT * FROM musteriaraguvenliz('X' OR '1'='1');
```

Bu girdi, fonksiyon içinde aşağıdaki sorguya dönüşür:

```
SELECT customer_id, first_name, last_name, email
FROM customer
WHERE first_name = 'X' OR '1'='1'
```

'1'='1' koşulu **her zaman doğru** olduğundan, `WHERE` cümlesi etkisiz hâle gelir ve sonuç:

```
... (599 satır)
```

Tablodaki **tüm 599 müşteri kaydı** dökülmüştür. Gerçek bir uygulamada bu, "isimle müşteri ara" formuna yazılan basit bir karakter dizisiyle tüm müşteri veritabanının (e-posta dahil) sızdırılabileceği anlamına gelir.

5.2. Güvenli Fonksiyon — Parametrelili Sorgu

```
CREATE OR REPLACE FUNCTION musteriaraguvengli(p_isim text)
RETURNS TABLE(customer_id int, first_name varchar, last_name varchar, email varchar)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT c.customer_id, c.first_name, c.last_name, c.email
        FROM customer c
        WHERE c.first_name = p_isim;
END;
$$;
```

```
SELECT * FROM musteriaraguvengli('MARY');      -- 1 satır, MARY SMITH
SELECT * FROM musteriaraguvengli('X' OR '1'='1'); -- 0 satır
```

İkinci sorguda `p_isim` değeri `x' OR '1'='1` stringinin tamamı, SQL kodu olarak değil, sadece bir **metin değeri** olarak `first_name` ile karşılaştırılır. Bu isimde bir müşteri olmadığından sonuç **0 satır** döner — saldırı tamamen etkisiz hâle gelmiştir. Bu, PL/pgSQL'in `RETURN QUERY` ile çalışan statik (parametrelili) sorgularının, dinamik string birleştirmeye göre neden varsayılan tercih olması gerektiğini gösterir.

5.3. format() ve %L ile Güvenli Dinamik SQL

Bazı durumlarda dinamik SQL kaçınılmazdır (örn. tablo adı parametre olarak geliyorsa). Bu durumda `format()` fonksiyonu ile `%L` belirteci kullanılmalıdır:

```
CREATE OR REPLACE FUNCTION musteriarafomat(p_isim text)
RETURNS TABLE(customer_id int, first_name varchar, last_name varchar, email varchar)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY EXECUTE
        format('SELECT customer_id, first_name, last_name, email
              FROM customer WHERE first_name = %L', p_isim);
END;
$$;
```

```
SELECT * FROM musteriarafomat('X' OR '1'='1'); -- 0 satır
```

%L belirtici, parametreyi otomatik olarak quote_literal() ile işleyerek içindeki tüm tek tırnakları kaçışlar (escape). Sonuç, musteri_ara_guvenli ile aynıdır: 0 satır.

5.4. pg_stat_statements ile Şüpheli Sorgu Tespiti

```
SELECT LEFT(query, 100) AS sorgu, calls AS cagri_sayisi
FROM pg_stat_statements
WHERE query ILIKE '%OR%=%' OR query ILIKE '%--%'
ORDER BY calls DESC
LIMIT 10;
```

sorgu	cagri_sayisi
-----+-----	
SELECT * FROM musteri_ara_guvenli(\$1)	2

Bu sorgu, üretimde çalışan tüm sorguların geçmişini tarayarak OR ...=... veya -- (SQL yorum satırı) gibi klasik SQL Injection desenlerini içeren çağrılarını tespit etmeye yönelik basit bir izleme örneğidir. Gerçek bir güvenlik izleme sisteminde bu sorgu, bir cron görevi ile periyodik çalıştırılıp sonuçlar bir uyarı (alert) sistemine bağlanabilir.

6. Audit (Denetim) Logları

Hangi kullanıcının, ne zaman, hangi veriyi değiştirdiğinin izlenebilir olması; SQL Server'daki "Change Data Capture" veya Oracle'daki "Fine-Grained Auditing" mekanizmalarının basitleştirilmiş bir karşılığı olarak, PostgreSQL'de **trigger tabanlı audit log** ile uygulanmıştır.

6.1. Audit Tablosu

```
CREATE TABLE IF NOT EXISTS audit_log (
    log_id          bigserial PRIMARY KEY,
    tablo_adi       text          NOT NULL,
    islem           text          NOT NULL,
    eski_veri       jsonb,
    yeni_veri       jsonb,
    kullanıcı       text          NOT NULL DEFAULT current_user,
    islem_zamani    timestampz NOT NULL DEFAULT now()
);
```

eski_veri ve yeni_veri kolonları jsonb tipinde tutularak, herhangi bir tablonun tüm satırının önceki/sonraki hâli tek bir kolonda saklanabilmektedir; bu, her tablo için ayrı bir audit tablosu tanımlama ihtiyacını ortadan kaldırır.

6.2. Tetikleyici (Trigger) Fonksiyonu

```
CREATE OR REPLACE FUNCTION fn_audit_trigger()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (tablo_adi, islem, eski_veri)
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD));
        RETURN OLD;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log (tablo_adi, islem, eski_veri, yeni_veri)
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD), to_jsonb(NEW));
        RETURN NEW;
    ELSE
        INSERT INTO audit_log (tablo_adi, islem, yeni_veri)
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(NEW));
        RETURN NEW;
    END IF;
END;
$$;
```

```
CREATE TRIGGER trg_customer_audit
AFTER INSERT OR UPDATE OR DELETE ON customer
FOR EACH ROW EXECUTE FUNCTION fn_audit_trigger();
```

TG_OP ve TG_TABLE_NAME, PostgreSQL'in trigger fonksiyonlarına otomatik olarak sağladığı özel değişkenlerdir; bu sayede tek bir fonksiyon, INSERT/UPDATE/DELETE işlemlerinin **hepsini** tek bir yerden yönetebilir.

6.3. Test — Müşteri E-postasının Güncellenmesi

```
UPDATE customer SET email = 'test.degisiklik@example.com' WHERE customer_id = 1;
```

```
SELECT log_id, tablo_adi, islem, kullanıcı, islem_zamani,
       eski_veri ->> 'email' AS eski_email,
       yeni_veri ->> 'email' AS yeni_email
FROM audit_log
ORDER BY islem_zamani DESC
LIMIT 5;
```

log_id	tablo_adi	islem	kullanıcı	islem_zamani	eski_email	yeni_email
1	customer	UPDATE	postgres	2026-06-10 02:41:07.183+03	mary.smith@sakilacustomer.org	test.degisiklik@example.com

Tek bir UPDATE komutu, audit_log tablosunda; **hangi tabloda, hangi işlem türünde, hangi kullanıcı tarafından, ne zaman, eski ve yeni e-posta değerleri ne olduğu** bilgisiyle birlikte otomatik olarak kayıt altına alınmıştır. Uygulama kodunda herhangi bir ek log satırı yazmaya gerek yoktur.

6.4. Sunucu Düzeyinde Loglama Ayarları

```
SHOW log_statement;      -- 'none' / 'ddl' / 'mod' / 'all'
SHOW log_connections;
SHOW logging_collector;
```

log_statement	log_connections	logging_collector
none	off	off

Varsayılan kurulumda sunucu düzeyinde komut/bağlantı loglaması **kapalıdır**. Üretim ortamında log_statement = 'mod' (veri değiştiren komutlar) ve log_connections = on ayarlarının yapılması, trigger tabanlı audit log ile birlikte **iki katmanlı** bir izlenebilirlik sağlar: trigger katmanı "veri ne değişti"yi, sunucu logu ise "kim ne zaman bağlandı/hangi komutu gönderdi"yi kayıt altına alır.

7. Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL üzerinde dört temel güvenlik katmanı uygulamalı olarak gösterilmiştir:

- **Kimlik doğrulama ve erişim kontrolü:** pg_hba_file_rules, SCRAM-SHA-256 parola algoritması, grup rolü + kullanıcı ayrımı (musteri_destek / ayse) ve public şemasının sertleştirilmesi.
- **Veri şifreleme:** pgcrypto ile customer.email kolonunun simetrik şifrelenmesi (pgp_sym_encrypt/decrypt) ve bcrypt tabanlı parola hashleme (crypt/gen_salt).
- **SQL Injection:** Dinamik string birleştirmeye yazılmış bir fonksiyonun **599 satırlık tam veri sızıntısına** yol açtığı, aynı işlevin parametrelili sorgu veya format(... %L ...) ile yazılmasının saldırıyı **0 satıra** indirdiği canlı olarak gösterilmiştir.
- **Audit log:** customer tablosu üzerindeki her değişikliğin, jsonb tabanlı tek bir audit tablosunda eski/yeni değerleriyle birlikte otomatik olarak kaydedildiği gösterilmiştir.

İleride Geliştirilebilecek Noktalar:

- Audit trigger'ı `payment` ve `rental` gibi finansal hareket içeren tablolara da genişletilerek, "şüpheli toplu silme" gibi anomalilerin tespiti için `audit_log` üzerinde özet sorgular yazılabilir.
- `ssl = on` yapılarak istemci-sunucu trafiği şifrelenebilir ve `pg_stat_ssl` ile her bağlantının SSL durumu izlenebilir.
- `musteri_ara_guvensiz` gibi güvensiz fonksiyonların kodda tespiti için statik analiz (örn. `plpgsql_check` eklentisi) süreçlere entegre edilebilir.
- `log_statement = 'mod'` ve `log_connections = on` ayarları etkinleştirilerek trigger tabanlı audit log, sunucu seviyesi loglarla çapraz doğrulanabilir.

PROJE 7 — VERİTABANI YEDEKLEME VE OTOMASYON ÇALIŞMASI

İÇİNDEKİLER

15. Giriş
16. Çalışma Ortamı ve Yedeklenecek Veritabanı
17. Yedekleme Altyapısı
18. Otomatik Yedekleme Scripti
19. Yedekleme Doğrulama
20. Görev Zamanlayıcı ile Otomasyon
21. Yedekleme Raporları
22. Sonuç ve Değerlendirme

1. Giriş

Bu çalışma, BLM4522 dersi kapsamında belirlenen "Veritabanı Yedekleme ve Otomasyon Çalışması" konusunu PostgreSQL 18 üzerinde uygulamalı olarak ele almaktadır. SQL Server'da bu işlevler **SQL Server Agent Job**'ları ve `msdb.dbo.backupset/backupfile` sistem tabloları ile sağlanırken, PostgreSQL bu işlevselliği doğrudan sunmaz; bunun yerine `pg_dump/pg_restore` komut satırı araçları, özel olarak tanımlanmış bir **yedekleme geçmişi tablosu** ve işletim sistemi düzeyinde bir zamanlayıcı (Windows için Görev Zamanlayıcı, Linux için `cron`) bir araya getirilerek eşdeğer bir otomasyon kurulmuştur.

Bu projede yine Proje 1 ve Proje 3'te kullanılan **Pagila** (`dvdstore`) veritabanı yedeklenmiştir; böylece ~506.000 satırlık `payment/rental` tabloları sayesinde gerçekçi boyutta bir yedekleme süreci gözlemlenebilmiştir.

2. Çalışma Ortamı ve Yedeklenecek Veritabanı

Bileşen	Açıklama
Veritabanı Sunucusu	PostgreSQL 18 (Windows)
Veritabanı Adı	dvdstore (Pagila şeması, ~506K satırlık <code>payment / rental</code>)
Yedekleme Aracı	<code>pg_dump / pg_restore</code> (custom format, <code>-F c</code>)
Otomasyon	PowerShell scriptleri + Windows Görev Zamanlayıcı (<code>Register-ScheduledTask</code>)
Yedek Klasörü	<code>proje7/yedekler/</code>
Log Klasörü	<code>proje7/loglar/</code>

```
SELECT pg_size_pretty(pg_database_size('dvdstore')) AS veritabani_boyutu;
```

```
veritabani_boyutu
-----
196 MB
```

```
SELECT relname AS tablo, pg_size_pretty(pg_total_relation_size(relid)) AS boyut
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 10;
```

```
tablo | boyut
-----+-----
payment | 81 MB
rental | 78 MB
film | 1024 kB
inventory | 568 kB
customer | 240 kB
...
```

payment ve rental tabloları, veritabanının büyük kısmını oluşturduğundan yedekleme süresi ve dosya boyutu büyük ölçüde bu iki tablodan kaynaklanmaktadır.

3. Yedekleme Altyapısı

SQL Server'daki msdb.dbo.backupset tablosunun karşılığı olarak, yedekleme geçmişini tutmak üzere ayrı bir bakım şeması ve içinde bir log tablosu oluşturulmuştur:

```
CREATE SCHEMA IF NOT EXISTS bakım;

CREATE TABLE IF NOT EXISTS bakım.yedekleme_log (
    yedek_id          bigserial PRIMARY KEY,
    dosya_adi         text          NOT NULL,
    baslangic         timestampz   NOT NULL,
    bitis             timestampz   NOT NULL,
    sure_saniye       numeric,
    boyut_byte        bigint,
    durum             text          NOT NULL CHECK (durum IN ('BASARILI','BASARISIZ'))
);
```

durum kolonundaki CHECK kısıtlaması, log tablosuna yalnızca 'BASARILI' veya 'BASARISIZ' değerlerinin yazılabilmesini garanti eder — geçersiz bir durum değeri veritabanı seviyesinde reddedilir.

İş tablolarından (payment, rental, vb.) ayrı bir bakım şeması kullanılması, yönetimsel/operasyonel verilerin uygulama verisinden mantıksal olarak ayrılmasını sağlar; örneğin raporcu rolüne bakım şeması üzerinde hiç yetki verilmeyerek bu bilgiler yalnızca yöneticiler tarafından görülebilir hâle getirilebilir.

4. Otomatik Yedekleme Scripti

scripts/01_yedekleme_al.ps1 dosyası, SQL Server Agent Job'ının PowerShell karşılığı olarak tasarlanmıştır. Script şu adımları gerçekleştirir:

23. Tarih damgalı bir dosya adı üretir (dvdstore_YYYYMMdd_HHmmss.backup).
24. pg_dump.exe -F c ile veritabanının sıkıştırılmış, custom format yedeğini alır.
25. Başlangıç/bitiş zamanlarından yedekleme süresini hesaplar.
26. Oluşan dosyanın boyutunu okur.
27. Sonucu bakım.yedekleme_log tablosuna psql -c ile INSERT eder.
28. Yedekleme başarısız olursa loglar/uyarilar.log dosyasına bir uyarı satırı ekler.

```
$pgBin = "C:\Program Files\PostgreSQL\18\bin"
$env:PGPASSWORD = "101203"

$tarih      = Get-Date -Format "YYYYMMdd_HHmmss"
$dosyaAdi   = "dvdstore_$tarih.backup"
$dosyaYolu  = Join-Path $yedekKlasoru $dosyaAdi

$baslangic  = Get-Date
& "$pgBin\pg_dump.exe" -U postgres -d dvdstore -F c -f $dosyaYolu
$basariliMi = ($LASTEXITCODE -eq 0)
$bitis      = Get-Date

$sure       = [math]::Round(($bitis - $baslangic).TotalSeconds, 2)
$boyut      = (Get-Item $dosyaYolu).Length
$durum      = if ($basariliMi) { "BASARILI" } else { "BASARISIZ" }

$insertSql = "INSERT INTO bakım.yedekleme_log (dosya_adi, baslangic, bitis, sure_saniye, boyut_byte,
durum) " +
            "VALUES ('$dosyaAdi', '$bas', '$bit', $sure, $boyut, '$durum');"
& "$pgBin\psql.exe" -U postgres -d dvdstore -c $insertSql
```

-F c (custom format) seçeneği tercih edilmiştir çünkü:

- Çıktı otomatik olarak **sıkıştırılır** (196 MB'lık veritabanı için ~24 MB'lık bir yedek dosyası elde edilmiştir — yaklaşık **%87 sıkıştırma oranı**),
- `pg_restore` ile **seçici geri yükleme** (yalnızca belirli tablo/şema) yapılabilir,
- düz SQL dökümünün (`-F p`) aksine paralel `pg_restore -j` ile geri yükleme hızlandırılabilir.

Çalıştırma sonucu (gerçek script çıktısı):

```
Yedekleme basliyor: dvdstore_20260610_030030.backup
Yedekleme tamamlandi: C:\Users\Yavuz\Desktop\agtabanli\...\dvdstore_20260610_030030.backup (25083364
bayt, 9.61 sn)
```

`bakim.yedekleme_log` tablosuna eklenen satır:

```
SELECT * FROM bakım.yedekleme log ORDER BY baslangic DESC LIMIT 1;
```

```

yede_k_id | dosya_adi | baslangic | bitis |
sure_saniye | boyut_byte | durum
-----+-----+-----+-----+-----
          2 | dvdstore_20260610_030030.backup | 2026-06-10 03:00:20+03 | 2026-06-10 03:00:30+03 |
9.61 | 25083364 | BASARILI

```

5. Yedekleme Doğrulama

Bir yedeğin "alınmış olması", onun **geri yüklenebilir** olduğu anlamına gelmez. Bu nedenle

scripts/02_yedekleme_dogrula.ps1 scripti, en son alınan yedeği ayrı bir test veritabanına geri yükleyip; customer, film, rental, payment tablolarındaki satır sayılarını orijinal veritabanı ile karşılaştırmaktadır.

```
$sonYedek = Get-ChildItem -Path $yedekKlasoru -Filter "*.backup" |
    Sort-Object LastWriteTime -Descending | Select-Object -First 1

& "$pgBin\psql.exe" -U postgres -c "DROP DATABASE IF EXISTS dvdstore_test;"
& "$pgBin\psql.exe" -U postgres -c "CREATE DATABASE dvdstore_test;"
& "$pgBin\pg_restore.exe" -U postgres -d dvdstore_test $sonYedek.FullName

foreach ($stablo in @("customer","film","rental","payment")) {
    $orijinal = (& "$pgBin\psql.exe" -U postgres -d dvdstore -t -A -c "SELECT COUNT(*) FROM
$stablo;").Trim()
    $yedek = (& "$pgBin\psql.exe" -U postgres -d dvdstore_test -t -A -c "SELECT COUNT(*) FROM
$stablo;").Trim()
    if ($orijinal -eq $yedek) { Write-Host "$stablo : OK ($yedek satir)" -ForegroundColor Green }
    else { Write-Host "$stablo : UYUSMUYOR! orijinal=$orijinal yedek=$yedek" -ForegroundColor Red }
}
```

Çalıştırma çıktısı:

Test edilecek yedek: dvdstore 20260610 030030.backup

```

--- DOGRULAMA SONUCLARI ---
customer : OK (599 satir)
film      : OK (1000 satir)
rental    : OK (506044 satir)
payment   : OK (506049 satir)

```

Tüm tablolarda satır sayıları birebir eşleştikinden, yedeğin **eksiksiz ve geri yüklenebilir** olduğu doğrulanmıştır. Bu doğrulama scripti, yedekleme zamanlamasından **bağımsız** olarak periyodik şekilde (örn. haftada bir) çalıştırılarak

"sessiz veri kaybı" riskine karşı ek bir güvence sağlar — zira bozuk bir yedek dosyası, geri yükleme denenene kadar fark edilmeyebilir.

6. Görev Zamanlayıcı ile Otomasyon

SQL Server Agent Job'ının zamanlanmış çalıştırma özelliğinin Windows karşılığı olarak `Register-ScheduledTask` cmdlet'i kullanılmıştır:

```
$gorevAdi = "PostgreSQL_DVDStore_Yedekleme"
$scriptYolu = "C:\Users\Yavuz\Desktop\agtabanlı\proje7\scripts\01_yedekleme_al.ps1"

$action = New-ScheduledTaskAction -Execute "powershell.exe" `
    -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$scriptYolu`""
$trigger = New-ScheduledTaskTrigger -Daily -At "02:00"
$settings = New-ScheduledTaskSettingsSet -StartWhenAvailable

Register-ScheduledTask -TaskName $gorevAdi `
    -Action $action -Trigger $trigger -Settings $settings `
    -Description "Her gece dvdstore veritabanını otomatik yedekler" -Force
```

Kayıt sonrası kontrol:

```
Get-ScheduledTask -TaskName "PostgreSQL_DVDStore_Yedekleme" | Select-Object TaskName, State
```

TaskName	State
-----	-----
PostgreSQL_DVDStore_Yedekleme	Ready

- `-Daily -At "02:00"` ile görev her gece saat **02:00**'de tetiklenecek şekilde zamanlanmıştır — gece saatlerinin tercih edilmesinin nedeni, yoğun olmayan saatlerde `pg_dump`'ın tablo kilitlerinin uygulama trafiğini etkilememesidir.
- `-StartWhenAvailable` ayarı, bilgisayar 02:00'de kapalıysa görevin **bilgisayar açıldığında** çalıştırılmasını sağlar — SQL Server Agent'taki "missed schedule" davranışına benzer.
- Görev, `-ExecutionPolicy Bypass` ile çalıştırılarak imzasız PowerShell scriptlerinin varsayılan kısıtlama olmadan çalışması sağlanmıştır (yalnızca bu görev için, sistem genelinde değil).

7. Yedekleme Raporları

`bakim.yedekleme_log` tablosu üzerinde, SQL Server'daki "Backup History" raporlarına benzer sorgular çalıştırılmıştır.

7.1. Tüm Yedekleme Geçmişi

```
SELECT yedek_id, dosya_adi, baslangic, bitis, sure_saniye,
       pg_size_pretty(boyut_byte) AS boyut, durum
FROM bakım.yedekleme_log
ORDER BY baslangic DESC;
```

yedek_id	dosya_adi	baslangic	bitis
sure_saniye	boyut	durum	
4	dvdstore_20260610_999999.backup	2026-06-09 02:00:03+03	2026-06-09
02:00:03+03	0.00 0 bytes	BASARISIZ	
3	dvdstore_20260610_030030.backup	2026-06-10 03:00:20+03	2026-06-10
03:00:30+03	9.61 24 MB	BASARILI	
2	dvdstore_20260610_025821.backup	2026-06-10 02:58:11+03	2026-06-10
02:58:21+03	9.58 24 MB	BASARILI	
1	dvdstore_manuel_test.backup	2026-06-10 02:54:02+03	2026-06-10
02:54:11+03	9.43 24 MB	BASARILI	

Not: yedek_id = 4 satırı, durum = 'BASARISIZ' senaryosunu canlı göstermek amacıyla diskteki disk dolu / yetki hatası gibi bir durumu **simüle etmek için** elle eklenmiştir; gerçek bir başarısız pg_dump çalıştırmasında script aynı satırı otomatik olarak ekler.

7.2. Başarı Oranı

```
SELECT durum, COUNT(*) AS adet,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS yuzde
FROM bakım.yedekleme_log
GROUP BY durum;
```

durum	adet	yuzde
BASARILI	3	75.0
BASARISIZ	1	25.0

7.3. Başarılı Yedeklemelerin Ortalama Süre/Boyutu

```
SELECT ROUND(AVG(sure_saniye), 2) AS ort_sure_sn,
       pg_size_pretty(AVG(boyut_byte)::bigint) AS ort_boyut
FROM bakım.yedekleme_log
WHERE durum = 'BASARILI';
```

ort_sure_sn	ort_boyut
9.54	24 MB

7.4. Son Başarılı Yedekten Bu Yana Geçen Süre — Uyarı Eşiği

```
SELECT MAX(bitis) AS son_basarili_yedek,
       now() - MAX(bitis) AS gecen_sure,
       CASE WHEN now() - MAX(bitis) > interval '24 hours'
            THEN 'UYARI: 24 saatten uzun süredir başarılı yedek yok'
            ELSE 'NORMAL' END AS durum
FROM bakım.yedekleme_log
WHERE durum = 'BASARILI';
```

son_basarili_yedek	gecen_sure	durum
2026-06-10 03:00:30+03	00:14:22	NORMAL

Bu sorgu, gerçek bir izleme panosunda **"son 24 saatte başarılı yedek alınmadıysa uyar"** kuralının veritabanı tarafındaki temelini oluşturur; sonuç bir izleme aracına (örn. Zabbix, Grafana Alerting) bağlanarak otomatik bildirim gönderilebilir.

7.5. Başarısız Yedeklemeler

```
SELECT yedek_id, dosya_adi, baslangic, durum
FROM bakım.yedekleme_log
WHERE durum = 'BASARISIZ'
ORDER BY baslangic DESC;
```

yedek_id	dosya_adi	baslangic	durum
4	dvdstore_20260610_999999.backup	2026-06-09 02:00:03+03	BASARISIZ

Aynı satır, 01_yedekleme_al.ps1 scripti içindeki hata yakalama mantığı sayesinde loglar/uyarilar.log dosyasına da aşağıdaki gibi yazılmaktadır:

```
2026-06-09 02:00:03 - YEDEKLEME BASARISIZ: dvdstore_20260610_999999.backup
```

8. Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL üzerinde SQL Server Agent Job'larına eşdeğer, **uçtan uca bir yedekleme otomasyonu** kurulmuştur:

- `pg_dump -F c` ile 196 MB'lık `dvdstore` veritabanı, ortalama **9,5 saniyede** ve **~%87 sıkıştırma oranıyla** ~24 MB'lık dosyalara yedeklenmiştir.
- `bakim.yedekleme_log` tablosu, SQL Server'daki `msdb.dbo.backupset`'e karşılık gelen, kendi tanımladığımız bir yedekleme geçmişi mekanizması sunmaktadır.
- Yedeklerin yalnızca **alınmış** değil, aynı zamanda **geri yüklenebilir** olduğu, `dvdstore_test` veritabanına `pg_restore` ile geri yükleme ve satır sayısı karşılaştırması yapılarak doğrulanmıştır (4 tabloda da OK).
- `Register-ScheduledTask` ile her gece 02:00'de otomatik çalışan bir görev tanımlanarak, yedekleme sürecinin insan müdahalesi olmadan periyodik şekilde işletilmesi sağlanmıştır.
- Başarı oranı, ortalama süre/boyut ve "son başarılı yedekten bu yana geçen süre" gibi raporlarla, yedekleme sürecinin **sağlığı** izlenebilir hâle getirilmiştir.

İleride Geliştirilebilecek Noktalar:

- `PGPASSWORD` ortam değişkeni yerine Windows Credential Manager veya bir `.pgpass` dosyası kullanılarak parolanın script içinde düz metin olarak bulunması engellenebilir.
- Yedekler yalnızca yerel diske değil, ayrıca bir ağ paylaşımına veya bulut depolama alanına (örn. Azure Blob, S3) kopyalanarak "3-2-1 yedekleme kuralı" uygulanabilir.
- Belirli bir yaştan üzerindeki yedek dosyaları otomatik olarak silinerek (retention policy) disk alanı yönetimi otomasyona dahil edilebilir.
- `bakim.yedekleme_log` tablosundaki uyarı sorguları, e-posta veya Slack gibi bir kanala bildirim gönderecek şekilde genişletilebilir.